Recursion on the Way Up - Study Guide

1. Understanding Recursion on the Way Up

Recursion on the way up refers to the process where the actual work or computation happens **after** the recursive call returns. In this pattern, we first make the recursive call, and then perform operations using the result returned from that call.

Key Characteristics:

- The recursive call is made first
- Processing happens when the call stack unwinds
- Results are built from the deepest level back to the surface
- The base case provides the starting value for computation

Basic Structure:

```java
public returnType recursiveMethod(parameters) {
    // Base case
    if (baseCondition) {
        return baseValue;
    }

    // Recursive call first
    returnType result = recursiveMethod(modifiedParameters);

    // Process the result (work happens here on the way up)
    return processResult(result);
}
```

Example 1: Factorial

```java
public static int factorial(int n) {
    // Base case
    if (n <= 1) {
        return 1;
    }

    // Recursive call first
    int result = factorial(n - 1);

    // Work happens on the way up
    return n * result;
}
```

Execution trace for factorial(4):

- factorial(4) calls factorial(3)
- factorial(3) calls factorial(2)
- factorial(2) calls factorial(1)
- factorial(1) returns 1
- factorial(2) returns 2 * 1 = 2
- factorial(3) returns 3 * 2 = 6
- factorial(4) returns 4 * 6 = 24

Example 2: Sum of Array Elements

```java
public static int sumArray(int[] arr, int index) {
    // Base case
    if (index >= arr.length) {
        return 0;
    }

    // Recursive call first
    int restSum = sumArray(arr, index + 1);

    // Work happens on the way up
    return arr[index] + restSum;
}
```

2. Need of Recursion on the Way Up

When to Use Recursion on the Way Up:

2.1 Building Results from Bottom to Top

When you need to construct a result that depends on values from deeper recursive calls:

```java
// Building a string in reverse order
public static String reverseString(String str, int index) {
    if (index >= str.length()) {
        return "";
    }

    String restReversed = reverseString(str, index + 1);
    return restReversed + str.charAt(index);
}
```

2.2 Accumulating Values

When you need to collect or accumulate results from all recursive calls:

```java
// Count total nodes in a binary tree
public static int countNodes(TreeNode root) {
    if (root == null) {
        return 0;
    }

    int leftCount = countNodes(root.left);
    int rightCount = countNodes(root.right);

    return 1 + leftCount + rightCount;
}
```

2.3 Mathematical Computations

For calculations where the result depends on the return value of recursive calls:

```java
// Calculate power using recursion
public static int power(int base, int exp) {
    if (exp == 0) {
        return 1;
    }

    int halfPower = power(base, exp / 2);

    if (exp % 2 == 0) {
        return halfPower * halfPower;
    } else {
        return base * halfPower * halfPower;
    }
}
```

2.4 Data Structure Traversal

When processing tree or graph structures where parent nodes need information from their children:

```java
// Find maximum value in binary tree
public static int findMax(TreeNode root) {
    if (root == null) {
        return Integer.MIN_VALUE;
    }

    int leftMax = findMax(root.left);
    int rightMax = findMax(root.right);

    return Math.max(root.val, Math.max(leftMax, rightMax));
}
```

Advantages:

- Natural for problems that require building results from smaller subproblems
- Clean and intuitive code structure
- Follows the mathematical definition of many recursive problems
- Easy to understand the flow of computation

Disadvantages:

- Uses more memory due to call stack
- Can be slower than iterative solutions
- Risk of stack overflow for deep recursions

3. Practice Questions

Question 1: Fibonacci Sequence

```java
// Calculate nth Fibonacci number using recursion on the way up
public static int fibonacci(int n) {
    if (n <= 1) {
        return n;
    }

    int fib1 = fibonacci(n - 1);
    int fib2 = fibonacci(n - 2);

    return fib1 + fib2;
}
```

Question 2: Binary Tree Height

```java
// Calculate height of binary tree
public static int treeHeight(TreeNode root) {
    if (root == null) {
        return 0;
    }

    int leftHeight = treeHeight(root.left);
    int rightHeight = treeHeight(root.right);

    return 1 + Math.max(leftHeight, rightHeight);
}
```

Question 3: Array Maximum Element

```java
// Find maximum element in array using recursion
public static int findArrayMax(int[] arr, int index) {
    if (index == arr.length - 1) {
        return arr[index];
    }

    int maxFromRest = findArrayMax(arr, index + 1);

    return Math.max(arr[index], maxFromRest);
}
```

Question 4: Count Digits

```java
// Count number of digits in a positive integer
public static int countDigits(int n) {
    if (n < 10) {
        return 1;
    }

    int restCount = countDigits(n / 10);

    return 1 + restCount;
}
```

Quiz Questions

1. What is the main characteristic of "recursion on the way up"? a) The base case is checked at the end b) Work is done before making the recursive call c) Work is done after the recursive call returns d) No base case is needed

Answer: c) Work is done after the recursive call returns

2. In the factorial function using recursion on the way up, when does the multiplication happen? a) Before the recursive call b) During the recursive call c) After the recursive call returns d) In the base case only

Answer: c) After the recursive call returns

3. Which of the following is NOT a good use case for recursion on the way up? a) Building results from smaller subproblems b) Simple iteration over an array c) Tree traversal requiring parent-child information flow d) Mathematical computations depending on recursive results

Answer: b) Simple iteration over an array

4. What happens first in recursion on the way up pattern? a) Processing the current element b) Returning the result c) Making the recursive call d) Checking the base case

Answer: d) Checking the base case

5. In recursion on the way up, the call stack unwinds: a) From top to bottom b) From bottom to top c) Randomly d) It doesn't unwind

Answer: b) From bottom to top

Practice Coding Questions

1. String Length Calculator Write a recursive function to calculate the length of a string using recursion on the way up pattern.

```java
public static int stringLength(String str, int index) {
    // Your code here
}
```

2. Sum of Digits Write a recursive function to calculate the sum of all digits in a positive integer using recursion on the way up.

```java
public static int sumOfDigits(int n) {
    // Your code here
}
```

3. Array Product Write a recursive function to calculate the product of all elements in an integer array using recursion on the way up.

```java
public static int arrayProduct(int[] arr, int index) {
    // Your code here
}
```

4. Binary Tree Sum Write a recursive function to calculate the sum of all node values in a binary tree using recursion on the way up.

```java
public static int treeSum(TreeNode root) {
    // Your code here
}
```

5. Palindrome Checker Write a recursive function to check if a string is a palindrome using recursion on the way up pattern.

```java
public static boolean isPalindrome(String str, int start, int end) {
    // Your code here
}
```